Aggregation

A UML aggregation is essentially a strong association. Like associations, an *aggregation* represents a class containing references to other classes. The key difference is that with an aggregation the referring class is only fully functional when it contains referred instances. Without the aggregation, there is no real need for the referring class to exist.

For example, imagine that our bookshops are now holding author signing events. We add an `Event` class to our model and associate `Customer` instances with it. The `Event` class has one main purpose: to organize the customers in the hopes that they will attend. To represent the aggregation of `Customer` instances into an `Event`, we use a line with an unfilled diamond like this:



Figure 10-9. *`Customer` and `Event` aggregation*

In this form of relationship, `Customer` instances can be associated with multiple `Event` instances. Cancelling an event and deleting its `Event` object in the system will not mean deleting any aggregated `Customer` objects.

If, however, a class is only ever associated with its containing class and cannot be referred to by any other, then the aggregation becomes *composition*.

Composition

Composition relationships are represented with a solid line and a filled diamond symbol. They indicate that the referred class can only be accessed via its container, and deleting a container instance will also delete its contents (because no more references to them will exist).

Our `Event` class could perhaps have its scheduling data encapsulated in, say, an `EventSchedule` object. `EventSchedule` might contain information like date and time, venue, and so on. The `EventSchedule` is so specific to an `Event` that its continued existence in the system should an `Event` be deleted would make no sense.